

Sang Ah Lee  
DATA-260 Section 22  
Student ID: 012634544

**Configuration:**

**SID4: 4544**

**PORT\_BASE: 8044**

**PREFIX: s4544**

**SEED: 4544**

**VERIFY\_SEED: 264544**

**DOMAIN\_ID: 4 (Open\_source package vulnerabilities)**

**Hardware: Apple M3, 16GB**

**Model: qwen2.5:3b (switched from qwen3.8b due to hardware issues)**

**Recent repo state (w/o report): ecd67e7de69f6b963be2d9be7ce2e1653ab35458**

**Link: <https://github.com/sangahlee0/data260-4544>**

**Link to HW5 Commit#:**

**<https://github.com/sangahlee0/data260-4544/tree/ecd67e7de69f6b963be2d9be7ce2e1653ab35458>** (This commit link does not include the report, because it will be added after downloading this document)

# Homework 5

## PART 1

### I. Database

```
# Domain Entity
class Vulnerability(Base):
    __tablename__ = "vulnerabilities"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    vulnerability_name = Column(String(255), nullable=False)

    vulnerability_code = Column(String(100), nullable=False, unique=True) # unique field
    # numeric field with sensible default
    count = Column(Integer, nullable=False, default=0)
    # foreign key related to entity (package_name is updated to package_id)
    package_id = Column(Integer, ForeignKey("packages.id"), nullable=False)
    # timestamps
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), server_default=func.now(), onupdate=func.now())
```

```
# Create second table for entity type
class Package(Base):
    __tablename__ = "packages"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    # primary text field
    name = Column(String(255), nullable=False, unique=True)
    # secondary text field
    version = Column(String(50), nullable=False)
    # unique field
    package_code = Column(String(100), nullable=False, unique=True)

    # timestamp fields
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), server_default=func.now(), onupdate=func.now())
```

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String(255), nullable=False)
    email = Column(String(255), nullable=False, unique=True)
    password_hash = Column(String(255), nullable=False)
```

### II. API Implementation

- Implement CRUD endpoints for package, and include pagination

```

def create_package(db: Session, payload: schema.PackageCreate):
    package = models.Package(name=payload.name, version=payload.version, package_code=payload.package_code)
    db.add(package)
    db.commit()
    db.refresh(package)
    return package

def get_packages(db: Session, skip: int = 0, limit: int = 15):
    return db.query(models.Package).order_by(models.Package.id.asc()).offset(skip).limit(limit).all()

def get_package(db: Session, package_id: int):
    return db.query(models.Package).filter(models.Package.id == package_id).first()

def update_package(db: Session, package_id: int, payload: schema.PackageUpdate):
    package = get_package(db, package_id)
    if not package:
        return None
    package.name = payload.name
    package.version = payload.version
    package.package_code = payload.package_code
    db.commit()
    db.refresh(package)
    return package

def delete_package(db: Session, package_id: int):
    package = get_package(db, package_id)
    if not package:
        return None
    vulnerabilities = get_vulnerabilities_by_package(db, package_id)

```

- Update for vulnerabilities, and retrieve with pagination

```

def create_vulnerability(db: Session, payload: schema.VulnerabilityCreate):
    vulnerability = models.Vulnerability(package_id=payload.package_id, vulnerability_name=payload.vulnerability_name)
    db.add(vulnerability)
    db.commit()
    db.refresh(vulnerability)
    return vulnerability

def get_vulnerabilities(db: Session, skip: int = 0, limit: int = 15):
    return db.query(models.Vulnerability).order_by(models.Vulnerability.id.asc()).offset(skip).limit(limit).all()

```

- Additional endpoint to get all primary-entity records

```

# Get all primary-entity records associated with a specific secondary-entity record
def get_vulnerabilities_by_package(db: Session, package_id: int):
    return db.query(models.Vulnerability).filter(models.Vulnerability.package_id == package_id).all()

```

- Create additional endpoint to get all primary-entity records associated with related-entity record

```

# Relationship endpoint
@app.get("/api/packages/{package_id}/vulnerabilities", response_model=list[schema.VulnerabilityOut])
def get_vulnerabilities_by_package(package_id: int, db: Session = Depends(get_db), _session=Depends(require_session)):
    """Get all vulnerabilities for a specific package - Returns JSON array of vulnerability objects"""
    package = crud.get_package(db, package_id)
    if not package:
        raise HTTPException(status_code=404, detail="Package not found")

    vulnerabilities = crud.get_vulnerabilities_by_package(db, package_id)
    return vulnerabilities

```

- Pydantic schema for request/response validation
  - schema.py

```

class VulnerabilityCreate(BaseModel):
    package_id: int
    vulnerability_name: str = Field(min_length=1)
    vulnerability_code: str = Field(min_length=1)
    urgency_score: int = Field(default=1, ge=1) # Ensure urgency score is a positive

class VulnerabilityUpdate(BaseModel):
    package_id: int
    vulnerability_name: str = Field(min_length=1)
    vulnerability_code: str = Field(min_length=1)
    urgency_score: int = Field(default=1, ge=1) # Ensure urgency score is a positive

class VulnerabilityOut(BaseModel):
    id: int
    package_id: int
    vulnerability_name: str
    vulnerability_code: str
    urgency_score: int

```

```

# Update pydantic schemas for the second entity (Package)
class PackageCreate(BaseModel):
    name: str = Field(min_length=1)
    version: str = Field(min_length=1)
    package_code: str = Field(min_length=3, pattern=r"^[A-Za-z0-9-]+$")

class PackageUpdate(BaseModel):
    name: str = Field(min_length=1)
    version: str = Field(min_length=1)
    package_code: str = Field(min_length=3, pattern=r"^[A-Za-z0-9-]+$")

class PackageOut(BaseModel):
    id: int
    name: str
    version: str
    package_code: str

class Config:
    from_attributes = True

```

- Proper HTTP status codes
  - Example code for resource-not-found, validation errors, and constraint violations

```

# Update vulnerability using the ID
@app.put("/api/vulnerabilities/{vulnerability_id}", response_model=schema.VulnerabilityOut)
def update_record(vulnerability_id: int, payload: schema.VulnerabilityUpdate, db: Session = Depends(get_db), _session=Depends(require_session)):
    """Update an existing vulnerability - Accepts JSON with vulnerability details"""
    # Check if the package exists before updating the vulnerability
    package = crud.get_package(db, payload.package_id)
    if not package:
        raise HTTPException(status_code=404, detail="Invalid package_id: Package does not exist")

    existing_vulnerability = db.query(models.Vulnerability).filter(models.Vulnerability.vulnerability_code == payload.vulnerability_code).first()
    if existing_vulnerability:
        raise HTTPException(status_code=409, detail="Vulnerability with this vulnerability_code already exists")

    vulnerability = crud.update_vulnerability(db, vulnerability_id, payload)
    if not vulnerability:
        raise HTTPException(status_code=404, detail="Vulnerability not found")

    return vulnerability

```

## - Postman screenshots

### - POST /auth/login

Postman interface for a POST request to `http://localhost:8044/auth/login`. The request is saved and shared. The response is a 200 status code, indicating success. The console shows two successful requests.

| Request                               | Response | Time     |
|---------------------------------------|----------|----------|
| POST http://localhost:8044/auth/login | 200      | 53.9 ms  |
| POST http://localhost:8044/auth/login | 200      | 239.2 ms |

### - POST /api/packages

Postman interface for a POST request to `http://localhost:8044/api/packages`. The request body is a JSON object: `{ "name": "requests", "version": "2.31.0", "package_code": "requests-231" }`. The response is a 201 status code, indicating that the resource was created. The console shows three successful requests.

| Request                                 | Response    | Time      |
|-----------------------------------------|-------------|-----------|
| POST http://localhost:8044/api/packages | 201 Created | 25 ms     |
| POST http://localhost:8044/api/packages | 401         | 25.39 ms  |
| POST http://localhost:8044/auth/login   | 200         | 104.24 ms |
| POST http://localhost:8044/api/packages | 201         | 24.7 ms   |

### - POST /api/packages

Postman interface for a POST request to `http://localhost:8044/api/packages`. The request body is a JSON object: `{ "name": "requests", "version": "2.31.0", "package_code": "requests-231" }`. The response is a 409 status code, indicating a conflict. The console shows four requests, including the conflict response.

| Request                                 | Response     | Time      |
|-----------------------------------------|--------------|-----------|
| POST http://localhost:8044/api/packages | 409 Conflict | 17 ms     |
| POST http://localhost:8044/api/packages | 401          | 25.39 ms  |
| POST http://localhost:8044/auth/login   | 200          | 104.24 ms |
| POST http://localhost:8044/api/packages | 201          | 24.7 ms   |
| POST http://localhost:8044/api/packages | 409          | 16.97 ms  |

## - GET /api/packages?skip=1&limit=1

POST login • POST api/packages • hw5 GET New Request + No Environment

hw5 > New Request

GET http://localhost:8044/api/packages?skip=1&limit=1 Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

none

This request does not have a body

Body 200 OK • 9 ms • 128 B Save Response

|                                                                                                                                                                                                                                                                                                                                                                                                                                              |          |        |          |       |      |      |         |       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|----------|-------|------|------|---------|-------|
| Console                                                                                                                                                                                                                                                                                                                                                                                                                                      | Terminal | Issues | All Logs | Clear | Copy | More | Refresh | Close |
| <div>▶ POST http://localhost:8044/api/packages 401 25.39 ms</div> <div>▶ POST http://localhost:8044/auth/login 200 104.24 ms</div> <div>▶ POST http://localhost:8044/api/packages 201 24.7 ms</div> <div>▶ POST http://localhost:8044/api/packages 409 16.97 ms</div> <div>▶ GET http://localhost:8044/api/packages?skip=1&amp;limit=1 200 30.02 ms</div> <div>▶ GET http://localhost:8044/api/packages?skip=1&amp;limit=1 200 8.97 ms</div> |          |        |          |       |      |      |         |       |

## - GET /api/packages/1

hw5 > packages/1

GET http://localhost:8044/api/packages/1 Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Body 200 OK • 20 ms • 202 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 1,
3   "name": "requests",
4   "version": "2.31.0",
5   "package_code": "requests-231"
6 }
```

Console Terminal Issues All Logs Clear Copy More Refresh Close

|                                                         |  |     |          |
|---------------------------------------------------------|--|-----|----------|
| ▶ POST http://localhost:8044/api/packages               |  | 201 | 24.7 ms  |
| ▶ POST http://localhost:8044/api/packages               |  | 409 | 16.97 ms |
| ▶ GET http://localhost:8044/api/packages?skip=1&limit=1 |  | 200 | 30.02 ms |
| ▶ GET http://localhost:8044/api/packages?skip=1&limit=1 |  | 200 | 8.97 ms  |
| ▶ POST http://localhost:8044/api/packages/1             |  | 405 | 9.34 ms  |
| ▶ GET http://localhost:8044/api/packages/1              |  | 200 | 20.47 ms |

## - POST /api/vulnerabilities

HTTP hw5 > api/vulnerabilities Save Share

**POST** ▼ http://localhost:8044/api/vulnerabilities Send ▼

Docs Params Authorization Headers 10 **Body** Scripts Settings Cookies

raw ▼ **JSON** ▼ Schema Beautify

```
1 {
2   "package_id": 2,
3   "vulnerability_name": "SQL Injection",
4   "vulnerability_code": "VULN-001",
5   "urgency_score": 1
6 }
```

Body ▼ ↺ 201 Created • 40 ms • 257 B • 🌐 Save Response ⋮

{} **JSON** ▼ Preview Visualize ▼ ↺ ≡ 🔍 📄 🔗

```
1 {
2   "id": 2,
3   "package_id": 2,
4   "vulnerability_name": "SQL Injection",
5   "vulnerability_code": "VULN-001",
6   "urgency_score": 1
7 }
```

## - GET /api/vulnerabilities

HTTP hw5 > api/vulnerabilities Save Share

**GET** ▼ http://localhost:8044/api/vulnerabilities Send

Docs Params Authorization Headers 8 **Body** Scripts Settings Cooki

none ▼

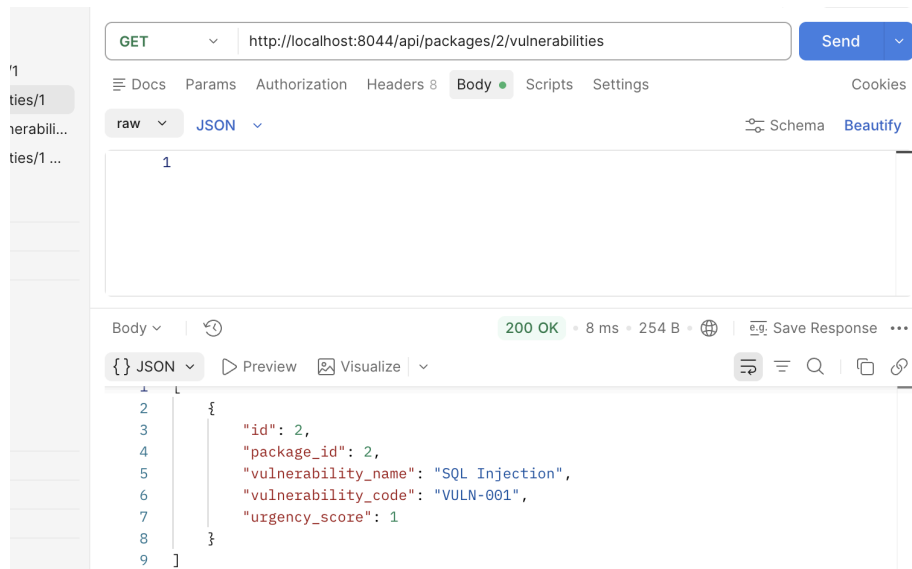
This request does not have a body

Body ▼ ↺ 200 OK • 17 ms • 398 B • 🌐 Save Response

{} **JSON** ▼ Preview Visualize ▼ ↺ ≡ 🔍 📄

```
4   "package_id": 2,
5   "package_name": "requests",
6   "vulnerability_name": "SQL Injection",
7   "vulnerability_code": "VULN-001",
8   "urgency_score": 1,
9   "descriptions": []
10  }
11 ]
```

## - GET /api/vulnerabilities/2



GET  Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

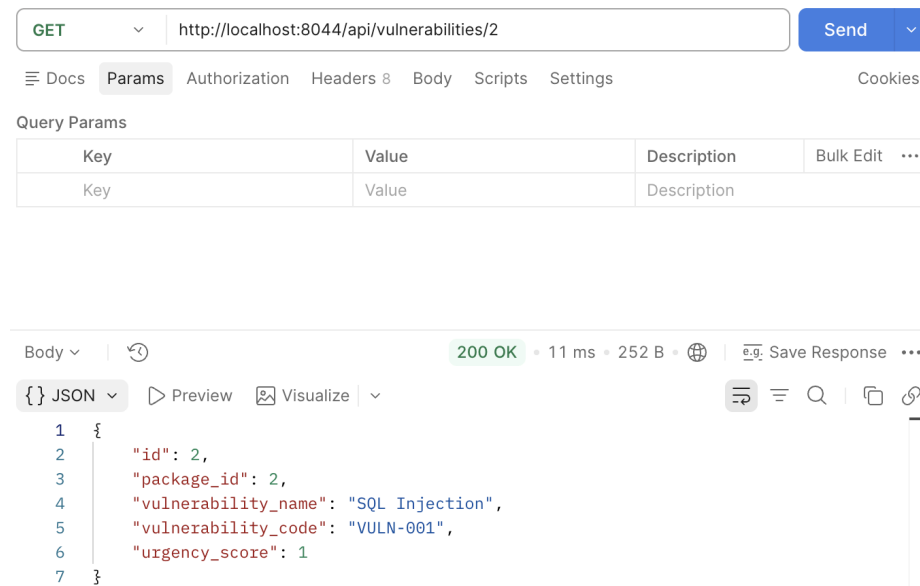
raw JSON Schema Beautify

```
1
```

Body 200 OK • 8 ms • 254 B • Save Response

```
{
  "id": 2,
  "package_id": 2,
  "vulnerability_name": "SQL Injection",
  "vulnerability_code": "VULN-001",
  "urgency_score": 1
}
```

## - PUT /api/vulnerabilities/2



GET  Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

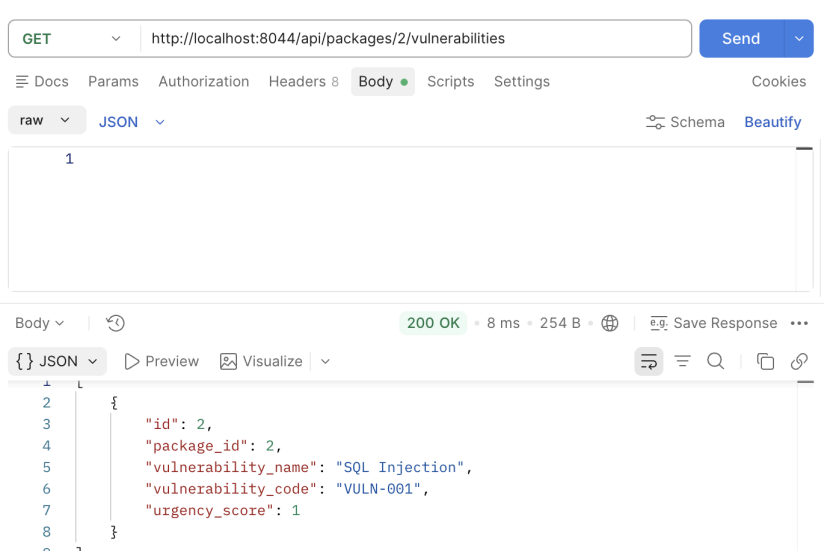
Query Params

| Key | Value | Description | Bulk Edit | ... |
|-----|-------|-------------|-----------|-----|
| Key | Value | Description |           |     |

Body 200 OK • 11 ms • 252 B • Save Response

```
{
  "id": 2,
  "package_id": 2,
  "vulnerability_name": "SQL Injection",
  "vulnerability_code": "VULN-001",
  "urgency_score": 1
}
```

- GET /api/packages/2/vulnerabilities



GET http://localhost:8044/api/packages/2/vulnerabilities

Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

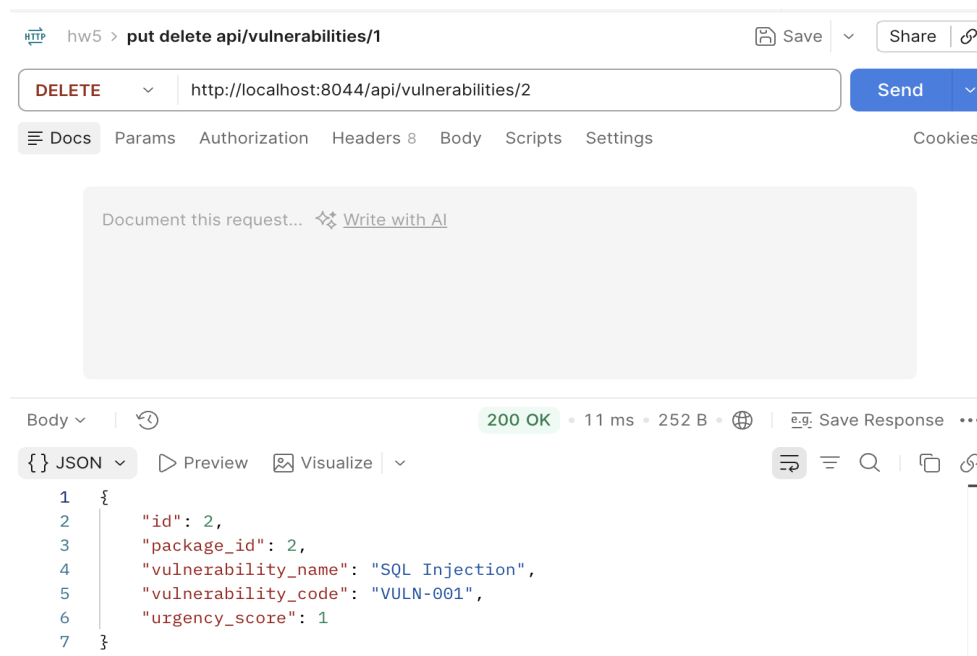
raw JSON Schema Beautify

1

Body 200 OK • 8 ms • 254 B Save Response

```
{
  "id": 2,
  "package_id": 2,
  "vulnerability_name": "SQL Injection",
  "vulnerability_code": "VULN-001",
  "urgency_score": 1
}
```

- DELETE /api/vulnerabilities/2



hw5 > put delete api/vulnerabilities/1

Save Share

DELETE http://localhost:8044/api/vulnerabilities/2

Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Document this request... Write with AI

Body 200 OK • 11 ms • 252 B Save Response

```
{
  "id": 2,
  "package_id": 2,
  "vulnerability_name": "SQL Injection",
  "vulnerability_code": "VULN-001",
  "urgency_score": 1
}
```

## - DELETE /api/packages/2

hw5 > put api/vulnerabilities/1 Copy 2

DELETE http://localhost:8044/api/packages/2

Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Body 200 OK 12 ms 216 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 2,
3   "name": "requests",
4   "version": "2.31.0",
5   "package_code": "requests-231"
6 }
```

```
mysql> SHOW TABLES;
+-----+
| Tables_in_s4544_rel |
+-----+
| packages             |
| sessions             |
| users               |
| vulnerabilities      |
| vulnerability_description |
+-----+
5 rows in set (0.00 sec)
```

```
mysql> DESC packages;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id     | int  | NO   | PRI | NULL    | auto_increment |
| name   | varchar(255) | NO | UNI | NULL    |
| version | varchar(50) | NO |     | NULL    |
| package_code | varchar(100) | NO | UNI | NULL    |
| created_at | datetime | YES |     | now()   | DEFAULT_GENERATED |
| updated_at | datetime | YES |     | now()   | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)
```

```
mysql> DESC sessions;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id     | varchar(64) | NO | PRI | NULL    |
| user_id | int | NO | MUL | NULL    |
| created_at | datetime | YES |     | now()   | DEFAULT_GENERATED |
| expires_at | datetime | NO |     | NULL    |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

```
mysql> DESC users;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id     | int  | NO   | PRI | NULL    | auto_increment |
| name   | varchar(255) | NO |     | NULL    |
| email  | varchar(255) | NO | UNI | NULL    |
| password_hash | varchar(255) | NO |     | NULL    |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> DESC vulnerability_description;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id     | int  | NO   | PRI | NULL    | auto_increment |
| vulnerability_id | int | NO | MUL | NULL    |
| description | varchar(255) | NO |     | NULL    |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> DESC vulnerabilities;
```

| Field              | Type         | Null | Key | Default | Extra             |
|--------------------|--------------|------|-----|---------|-------------------|
| id                 | int          | NO   | PRI | NULL    | auto_increment    |
| vulnerability_name | varchar(255) | NO   |     | NULL    |                   |
| vulnerability_code | varchar(100) | NO   | UNI | NULL    |                   |
| urgency_score      | int          | NO   |     | 1       |                   |
| package_id         | int          | NO   | MUL | NULL    |                   |
| created_at         | datetime     | YES  |     | now()   | DEFAULT_GENERATED |
| updated_at         | datetime     | YES  |     | now()   | DEFAULT_GENERATED |

7 rows in set (0.00 sec)

### III. Redux Client

#### 1. Redux Setup

```
code > web_application > frontend > store > JS store.js > ...
1 import { configureStore } from "@reduxjs/toolkit";
2 import vulnerabilitiesReducer from "../features/vulnerabilities/vulnerabilitiesSlice";
3
4 export const store = configureStore({
5   reducer: { vulnerabilities: vulnerabilitiesReducer }
6 });
```

```
code > web_application > frontend > features > vulnerabilities > JS vulnerabilitiesSlice.js > vulnerabilities
1 import { createSlice } from "@reduxjs/toolkit";
2
3 const vulnerabilitiesSlice = createSlice({
4   name: "vulnerabilities",
5   initialState: { items: [], loading: false, error: null },
6   reducers: {}
7 });
8
9 export default vulnerabilitiesSlice.reducer;
```

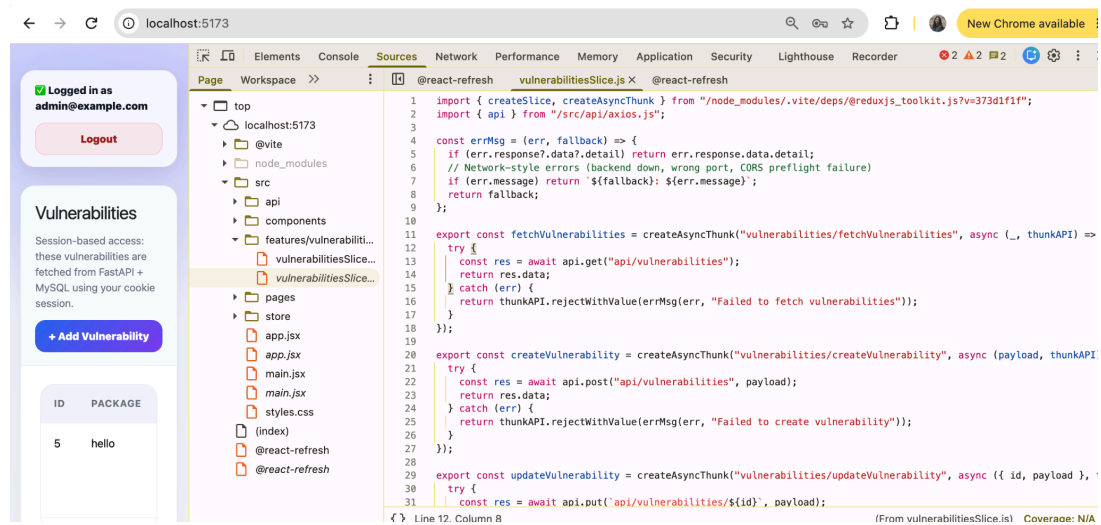
#### 2. Async Thunks with Axios

```
export const fetchVulnerabilities = createAsyncThunk("vulnerabilities/fetchVulnerabilities", async (_, thunkAPI) => {
  try {
    const res = await api.get("api/vulnerabilities");
    return res.data;
  } catch (err) {
    return thunkAPI.rejectWithValue(errMsg(err, "Failed to fetch vulnerabilities"));
  }
});

export const createVulnerability = createAsyncThunk("vulnerabilities/createVulnerability", async (payload, thunkAPI) => {
  try {
    const res = await api.post("api/vulnerabilities", payload);
    return res.data;
  } catch (err) {
    return thunkAPI.rejectWithValue(errMsg(err, "Failed to create vulnerability"));
  }
});

export const updateVulnerability = createAsyncThunk("vulnerabilities/updateVulnerability", async ({ id, payload }, thunkAPI) => {
  try {
    const res = await api.put(`api/vulnerabilities/${id}`, payload);
    return res.data;
  } catch (err) {
    return thunkAPI.rejectWithValue(errMsg(err, "Failed to update vulnerability"));
  }
});
```

### 3. Home Screen



Logged in as admin@example.com

Logout

#### Vulnerabilities

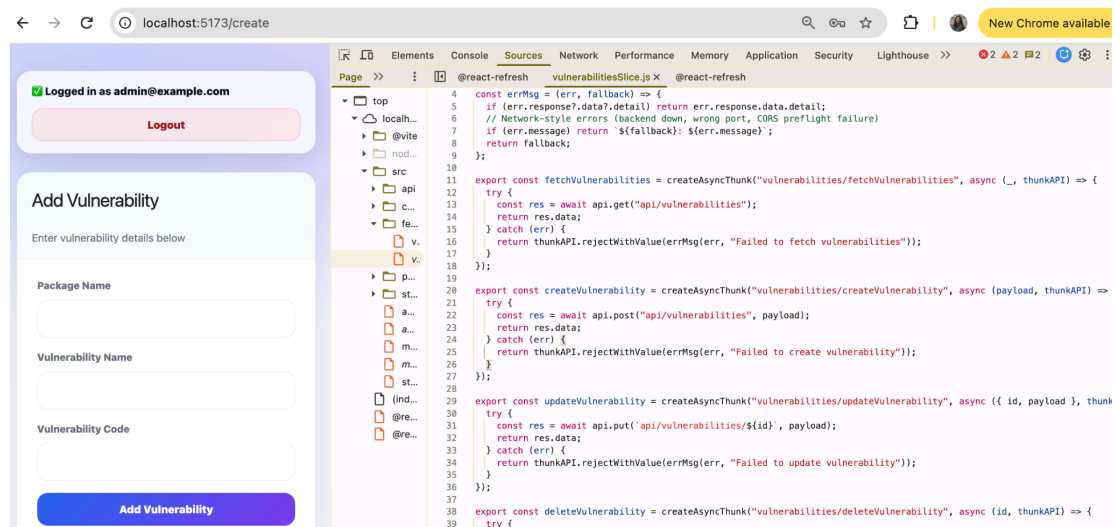
Session-based access: these vulnerabilities are fetched from FastAPI + MySQL using your cookie session.

+ Add Vulnerability

| ID | PACKAGE |
|----|---------|
| 5  | hello   |

```
1 import { createSlice, createAsyncThunk } from "/node_modules/.vite/deps/@reduxjs/toolkit.js?v=373d1f1f";
2 import { api } from "/src/api/axios.js";
3
4 const errMsg = (err, fallback) => {
5   if (err.response?.data?.detail) return err.response.data.detail;
6   // Network-style errors (backend down, wrong port, CORS preflight failure)
7   if (err.message) return `${fallback}: ${err.message}`;
8   return fallback;
9 };
10
11 export const fetchVulnerabilities = createAsyncThunk("vulnerabilities/fetchVulnerabilities", async (_, thunkAPI) => {
12   try {
13     const res = await api.get("api/vulnerabilities");
14     return res.data;
15   } catch (err) {
16     return thunkAPI.rejectWithValue(errMsg(err, "Failed to fetch vulnerabilities"));
17   }
18 });
19
20 export const createVulnerability = createAsyncThunk("vulnerabilities/createVulnerability", async (payload, thunkAPI) => {
21   try {
22     const res = await api.post("api/vulnerabilities", payload);
23     return res.data;
24   } catch (err) {
25     return thunkAPI.rejectWithValue(errMsg(err, "Failed to create vulnerability"));
26   }
27 });
28
29 export const updateVulnerability = createAsyncThunk("vulnerabilities/updateVulnerability", async ({ id, payload }, thunkAPI) => {
30   try {
31     const res = await api.put(`api/vulnerabilities/${id}`, payload);
32     return res.data;
33   } catch (err) {
34     return thunkAPI.rejectWithValue(errMsg(err, "Failed to update vulnerability"));
35   }
36 });
37
38 export const deleteVulnerability = createAsyncThunk("vulnerabilities/deleteVulnerability", async (id, thunkAPI) => {
39   try {
40     const res = await api.delete(`api/vulnerabilities/${id}`);
41     return res.data;
42   } catch (err) {
43     return thunkAPI.rejectWithValue(errMsg(err, "Failed to delete vulnerability"));
44   }
45 });
```

### 4. Create/Update Screens



Logged in as admin@example.com

Logout

#### Add Vulnerability

Enter vulnerability details below

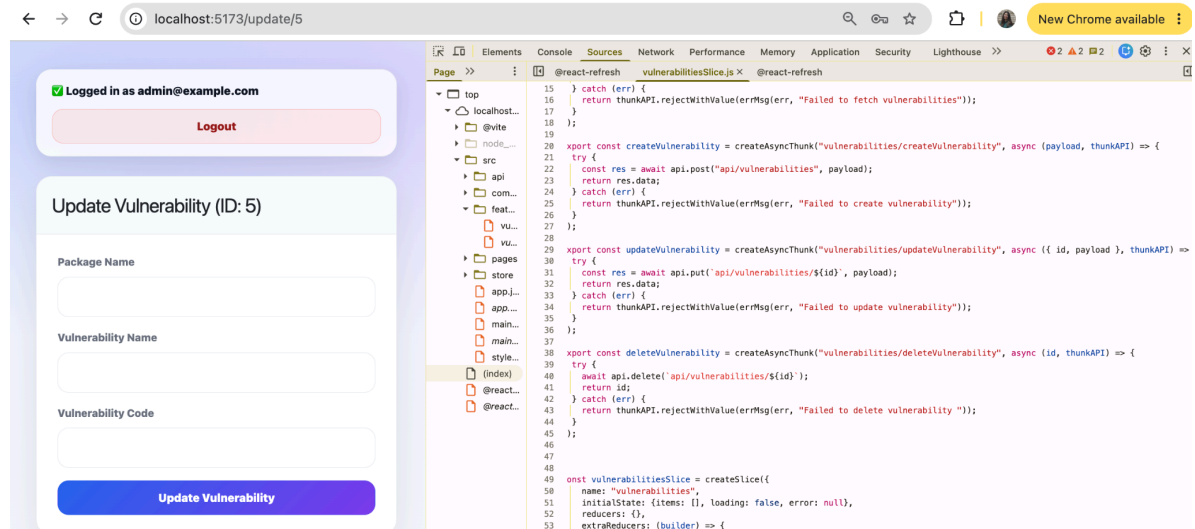
Package Name

Vulnerability Name

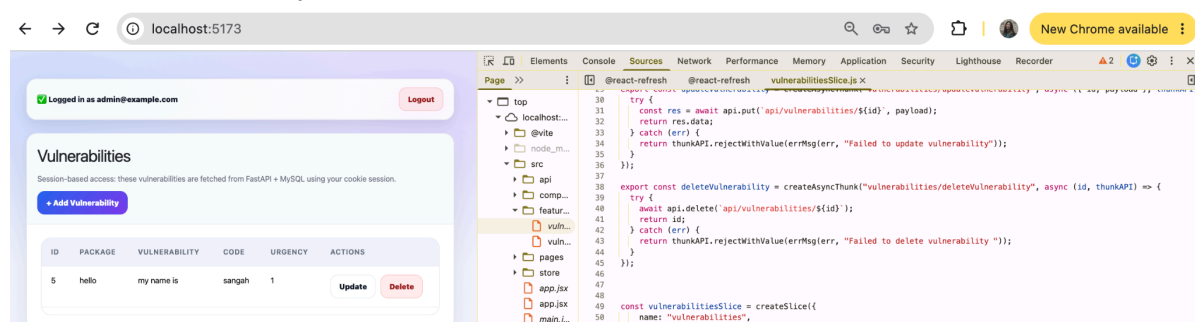
Vulnerability Code

Add Vulnerability

```
4 const errMsg = (err, fallback) => {
5   if (err.response?.data?.detail) return err.response.data.detail;
6   // Network-style errors (backend down, wrong port, CORS preflight failure)
7   if (err.message) return `${fallback}: ${err.message}`;
8   return fallback;
9 };
10
11 export const fetchVulnerabilities = createAsyncThunk("vulnerabilities/fetchVulnerabilities", async (_, thunkAPI) => {
12   try {
13     const res = await api.get("api/vulnerabilities");
14     return res.data;
15   } catch (err) {
16     return thunkAPI.rejectWithValue(errMsg(err, "Failed to fetch vulnerabilities"));
17   }
18 });
19
20 export const createVulnerability = createAsyncThunk("vulnerabilities/createVulnerability", async (payload, thunkAPI) => {
21   try {
22     const res = await api.post("api/vulnerabilities", payload);
23     return res.data;
24   } catch (err) {
25     return thunkAPI.rejectWithValue(errMsg(err, "Failed to create vulnerability"));
26   }
27 });
28
29 export const updateVulnerability = createAsyncThunk("vulnerabilities/updateVulnerability", async ({ id, payload }, thunkAPI) => {
30   try {
31     const res = await api.put(`api/vulnerabilities/${id}`, payload);
32     return res.data;
33   } catch (err) {
34     return thunkAPI.rejectWithValue(errMsg(err, "Failed to update vulnerability"));
35   }
36 });
37
38 export const deleteVulnerability = createAsyncThunk("vulnerabilities/deleteVulnerability", async (id, thunkAPI) => {
39   try {
40     const res = await api.delete(`api/vulnerabilities/${id}`);
41     return res.data;
42   } catch (err) {
43     return thunkAPI.rejectWithValue(errMsg(err, "Failed to delete vulnerability"));
44   }
45 });
```



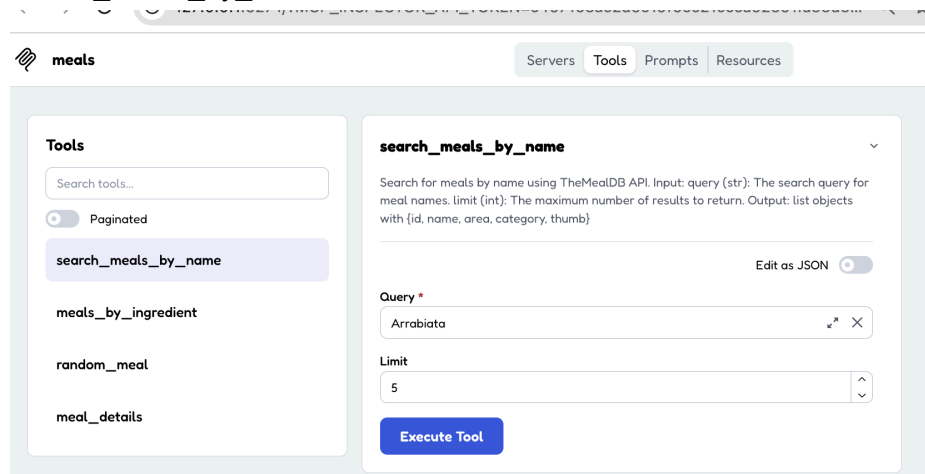
## 5. Delete Functionality



## PART 2 - MCP Tool servers

### A - TheMealDB Tutorial Server

#### search\_meals\_by\_name



meals

Servers Tools Prompts Resources

Tools

Search tools...

Paginated

search\_meals\_by\_name

meals\_by\_ingredient

random\_meal

meal\_details

Results

```
1- {
2  "id": "52771",
3  "name": "Spicy Arrabiata Penne",
4  "area": "Italian",
5  "category": "Vegetarian",
6  "thumb": "https://www.themeadb.com/images/media/meals/ustsqe14682506",
7 }
```

Protocol Console Search...

Messages LEGACY

Newest First Clear

Export

21:20:02 CLIENT - SERVER

TOOLS/CALL search\_meals\_by\_name

21:18:17 CLIENT - SERVER

```
code > mealdb_mcp > meals_server.py > search_meals_by_name
9 def search_meals_by_name(query: str, limit: int=5):
16
17     Output:
18         list objects with {id, name, area, category, thumb}
19
20     """
21     if limit < 1 or limit > 25:
22         return {"error": "Limit must be between 1 and 25."}
23
24     try:
25         response = requests.get(f"{BASE_URL}/search.php?s={query}")
26         response.raise_for_status() # Raise an exception for HTTP errors
27         data = response.json()
28     except requests.RequestException as e:
29         raise RuntimeError(f"Failed to fetch data from API: {e}")
30     except ValueError as e:
31         raise RuntimeError(f"Failed to parse JSON response: {e}")
32
33     meals = data.get("meals", [])
34     if not meals:
35         return {"message": "No matches found."}
36     results = []
37     for meal in meals[:limit]:
38         # Return only the required fields: id, name, area, category, and thumb
39         results.append({
40             "id": meal.get("idMeal"),
41             "name": meal.get("strMeal"),
42             "area": meal.get("strArea"),
43             "category": meal.get("strCategory"),
44             "thumb": meal.get("strMealThumb")
45         })
46     return results
```

## meals\_by\_ingredient

meals

Servers Tools Prompts Resources

Tools

Search tools...

Paginated

search\_meals\_by\_name

meals\_by\_ingredient

random\_meal

meal\_details

meals\_by\_ingredient

Input: ingredient (str). The ingredient to search for. limit (int). The maximum number of results to return. Output: list objects with {id, name, thumb}

Edit as JSON

Ingredient \*

beef

Limit

12

Execute Tool

Protocol Console Search...

Messages LEGACY

Newest First Clear

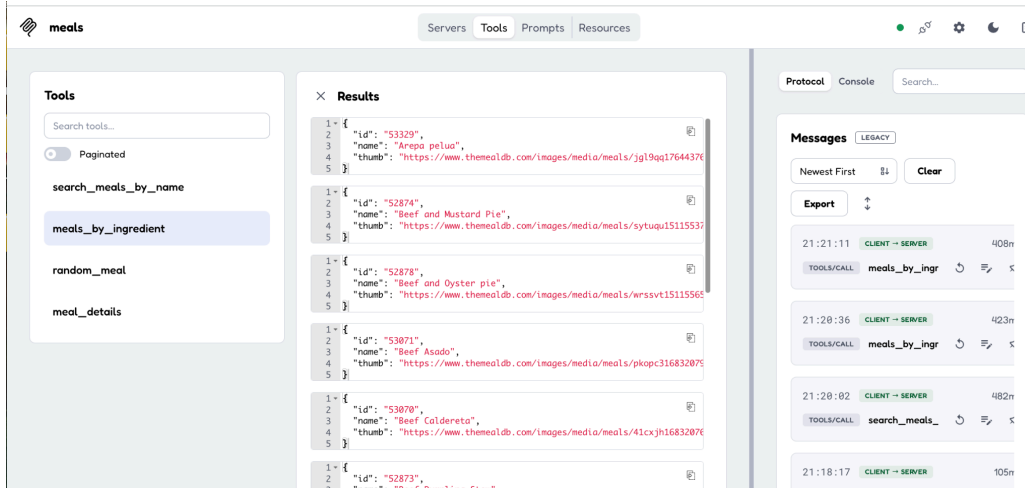
Export

21:20:36 CLIENT - SERVER 423ms

TOOLS/CALL meals\_by\_ingredient

21:20:02 CLIENT - SERVER 482ms

TOOLS/CALL search\_meals\_by\_name



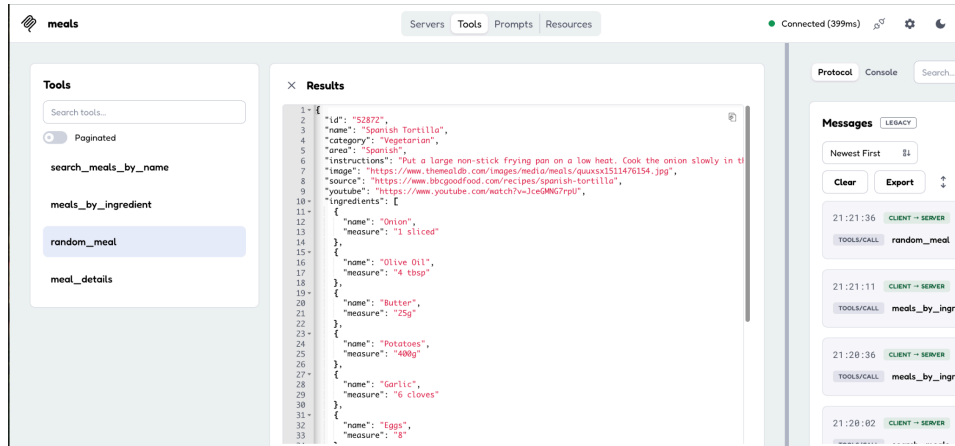
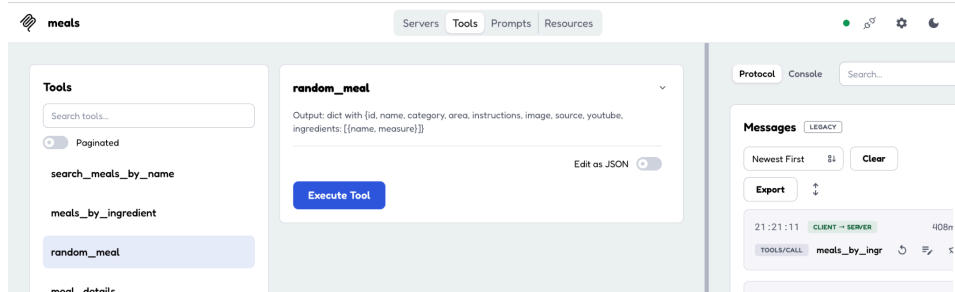
```
@mcp.tool()
def meals_by_ingredient(ingredient: str, limit: int=12):
    """
    Input:
        ingredient (str): The ingredient to search for.
        limit (int): The maximum number of results to return.

    Output:
        list objects with {id, name, thumb}

    """
    try:
        response = requests.get(f"{BASE_URL}/filter.php?i={ingredient}")
        response.raise_for_status()
        data = response.json()
    except requests.RequestException as e:
        raise RuntimeError(f"Failed to fetch data from API: {e}")
    except ValueError as e:
        raise RuntimeError(f"Failed to parse JSON response: {e}")

    meals = data.get("meals", [])
    if not meals:
        return {"message": "No matches found."}
    results = []
    for meal in meals[:limit]:
        results.append({
            "id": meal.get("idMeal"),
            "name": meal.get("strMeal"),
            "thumb": meal.get("strMealThumb")
        })
    return results
```

# random\_meal



```
mealsdb_mcp > meals_server.py > search_meals_by_name

@mcp.tool()
def random_meal():
    """
    Output:
    | dict with (id, name, category, area, instructions, image, source, youtube, ingredients: [{name, measure}])
    """
    try:
        response = requests.get(f"{BASE_URL}/random.php")
        response.raise_for_status()
        data = response.json()
    except requests.RequestException as e:
        raise RuntimeError(f"Failed to fetch data from API: {e}")
    except ValueError as e:
        raise RuntimeError(f"Failed to parse JSON response: {e}")

    meal = data.get("meals")

    if not meal:
        return {"message": "No matches found."}

    meal = meal[0] # Get the first meal from the list

    # Ingredients is a list of dicts with name and measure
    ingredients = []
    for i in range(1, 21): # db ingredient field is 1-20
        ingredient = meal.get(f"strIngredient{i}")
        measure = meal.get(f"strMeasure{i}")
        if ingredient and ingredient.strip():
            ingredients.append({"name": ingredient.strip(), "measure": measure.strip() if measure else ""})
    return {"id": meal.get("idMeal"),
            "name": meal.get("strMeal"),
```

## meal\_details

The screenshot shows the MCP client interface with the 'meals' server connected. The 'Tools' panel on the left lists several tools, with 'meal\_details' selected. The main panel shows the 'meal\_details' tool description: 'Get meal details by ID using TheMealDB API. Input: id (str). The ID of the meal. Output: dict with {id, name, category, area, instructions, image, source, youtube, ingredients: [{name, measure}]}'. An input field contains '53329' and an 'Execute Tool' button is visible. The right panel shows the 'Messages' log with a recent message from the client to the server.

The screenshot shows the MCP client interface with the 'meal\_details' tool results displayed. The 'Results' panel shows a JSON object with the following structure: 

```
{  "id": "53329",  "name": "Arepá pelúa",  "category": "Beef",  "area": "Venezuela",  "instructions": "Cook the meat: Place the flank steak in a pot with broth or water and salt",  "image": "https://www.themealdb.com/images/media/meals/jgl9qg1764437635.jpg",  "source": "https://goya.es/en/recipes/arepa-pelua",  "youtube": "https://www.youtube.com/watch?v=b5W0yWtGfo",  "ingredients": [    {      "name": "Beef",      "measure": "500g"    },    {      "name": "Onion",      "measure": "1"    },    {      "name": "Red Pepper",      "measure": "1"    },    {      "name": "Garlic",      "measure": "2 cloves"    },    {      "name": "Cumin",      "measure": "1 tsp"    },    {      "name": "Oregano",      "measure": "1 tsp"    }  ]}
```

```
@mcp.tool()
def meal_details(id: str):
    """
    Get meal details by ID using TheMealDB API.

    Input:
        id (str): The ID of the meal.

    Output:
        dict with {id, name, category, area, instructions, image, source, youtube, ingredients: [{name, measure}]}

    """
    try:
        response = requests.get(f"{BASE_URL}/lookup.php?i={id}")
        response.raise_for_status()
        data = response.json()
    except requests.RequestException as e:
        raise RuntimeError(f"Failed to fetch data from API: {e}")
    except ValueError as e:
        raise RuntimeError(f"Failed to parse JSON response: {e}")

    meal = data.get("meals")
    if not meal:
        return {"message": "No matches found."}
    meal = meal[0] # Get the first meal from the list
    # Ingredients is a list of dicts with name and measure
    ingredients = []
    for i in range(1, 21): # db ingredient field is 1-20
        ingredient = meal.get(f"strIngredient{i}")
        measure = meal.get(f"strMeasure{i}")
        if ingredient and ingredient.strip():
            ingredients.append({"name": ingredient.strip(), "measure": measure.strip() if measure else ""})
```

## B - Domain MCP Server

search - search\_vulnerabilities\_by\_name

vulnerabilities

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search\_vulnerabilities\_by\_name

search\_vulnerabilities\_by\_id

count\_by\_package

search\_vulnerabilities\_by\_name

Input: query (str): The search query for vulnerability names. limit (int): The maximum number of results to return.

Query \*  
Injection

Limit  
5

Execute Tool

ProtocolConsole

Search...

Messages

Newest First

Export

21:55:27 CLIENT → SER... 127ms

21:55:27 CLIENT → SER... 117ms

vulnerabilities

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search\_vulnerabilities\_by\_name

search\_vulnerabilities\_by\_id

count\_by\_package

Results

```
1 {
2   "ok": true,
3   "data": [
4     {
5       "id": 10,
6       "vulnerability_name": "SQL Injection Vulnerability",
7       "vulnerability_code": "VULN-DJA-001",
8       "urgency_score": 5,
9       "package_id": 11
10    }
11  ],
12  "error": null
13 }
```

vulnerabilities

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search\_vulnerabilities\_by\_name

search\_vulnerabilities\_by\_id

count\_by\_package

search\_vulnerabilities\_by\_name

Input: query (str): The search query for vulnerability names. limit (int): The maximum number of results to return.

Query \*  
zzz

Limit  
5

Execute Tool

Tools

Search tools...

Paginated

search\_vulnerabilities\_by\_name

×

Results

1

{

2

"ok": false,

3

"data": null,

4

"error": "No vulnerabilities found matching 'zzz'"

5

}

```
@mcp.tool()
def search_vulnerabilities_by_name(query: str, limit: int=5):
    """
    Input:
        query (str): The search query for vulnerability names.
        limit (int): The maximum number of results to return.
    """
    db = SessionLocal()
    try:
        vulnerabilities = (db.query(Vulnerability)
                           .filter(Vulnerability.vulnerability_name.ilike(f"%{query}%"))
                           .limit(limit)
                           .all())
        if not vulnerabilities:
            return {
                "ok": False,
                "data": None,
                "error": f"No vulnerabilities found matching '{query}'"
            }

        results = []
        for vulnerability in vulnerabilities:
            results.append({
                "id": vulnerability.id,
                "vulnerability_name": vulnerability.vulnerability_name,
                "vulnerability_code": vulnerability.vulnerability_code,
                "urgency_score": vulnerability.urgency_score,
                "package_id": vulnerability.package_id
            })

        return {"ok" : True,
                "data" : results,
```

## detail lookup - search\_vulnerabilities\_by\_id

vulnerabilities

Servers

Tools

Prompts

Resources

Tools

Search tools...

Paginated

search\_vulnerabilities\_by\_name

search\_vulnerabilities\_by\_id

count\_by\_package

search\_vulnerabilities\_by\_id

▼

Input: vulnerability\_id (int): The ID of the vulnerability to search for.

Edit as JSON

Vulnerability Id \*

10

^

v

Execute Tool

 vulnerabilities

ServersToolsPromptsResources

Tools

Paginated

search\_vulnerabilities\_by\_name

search\_vulnerabilities\_by\_id

count\_by\_package

× Results

```
1- {
2  "ok": true,
3  "data": {
4    "id": 10,
5    "vulnerability_name": "SQL Injection Vulnerability",
6    "vulnerability_code": "VULN-DJA-001",
7    "urgency_score": 5,
8    "package_id": 11
9  },
10 "error": null
11 }
```

 vulnerabilities

ServersToolsPromptsResources

Tools

Paginated

search\_vulnerabilities\_by\_name

search\_vulnerabilities\_by\_id

count\_by\_package

search\_vulnerabilities\_by\_id

Input: vulnerability\_id (int): The ID of the vulnerability to search for.

Edit as JSON

Vulnerability Id \*

999

Execute Tool

 vulnerabilities

ServersToolsPromptsResources

Tools

Paginated

search\_vulnerabilities\_by\_name

search\_vulnerabilities\_by\_id

count\_by\_package

× Results

```
1- {
2  "ok": false,
3  "data": null,
4  "error": "No vulnerability found with ID 999"
5 }
```

```
@mcp.tool()
def search_vulnerabilities_by_id(vulnerability_id: int):
    """
    Input:
        vulnerability_id (int): The ID of the vulnerability to search for.
    """
    db = SessionLocal()
    try:
        vulnerability = (db.query(Vulnerability)
                        .filter(Vulnerability.id == vulnerability_id)
                        .first())

        if not vulnerability:
            return {"ok": False,
                    "data": None,
                    "error": f"No vulnerability found with ID {vulnerability_id}"}

        return {"ok": True,
                "data": {
                    "id": vulnerability.id,
                    "vulnerability_name": vulnerability.vulnerability_name,
                    "vulnerability_code": vulnerability.vulnerability_code,
                    "urgency_score": vulnerability.urgency_score,
                    "package_id": vulnerability.package_id
                },
                "error": None}
    finally:
        db.close()
```

aggregate operation - count\_by\_package

 vulnerabilities

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search\_vulnerabilities\_by\_name

search\_vulnerabilities\_by\_id

count\_by\_package

count\_by\_package

Count the number of vulnerabilities associated with a specific package. Input: package\_name (str):  
The name of the package to search for.

Package Name \*

requests

Execute Tool

 vulnerabilities

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search\_vulnerabilities\_by\_name

search\_vulnerabilities\_by\_id

count\_by\_package

Results

1 {

2 "ok": true,

3 "data": 1,

4 "error": null

5 }

 vulnerabilities

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search\_vulnerabilities\_by\_name

search\_vulnerabilities\_by\_id

count\_by\_package

count\_by\_package

Count the number of vulnerabilities associated with a specific package. Input: package\_name (str):  
The name of the package to search for.

Package Name \*

jelly

Execute Tool

 vulnerabilities

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search\_vulnerabilities\_by\_name

search\_vulnerabilities\_by\_id

count\_by\_package

Results

1 {

2 "ok": false,

3 "data": null,

4 "error": "No package found with name 'jelly'"

5 }

```

@mcp.tool()
def count_by_package(package_name: str):
    """
    Count the number of vulnerabilities associated with a specific package.

    Input:
    | package_name (str): The name of the package to search for.
    """
    db = SessionLocal()
    try:
        package = (db.query(Package)
                    .filter(Package.name == package_name)
                    .first())

        if not package:
            return {
                "ok": False,
                "data": None,
                "error": f"No package found with name '{package_name}'"
            }

        count = (db.query(Vulnerability)
                  .filter(Vulnerability.package_id == package.id)
                  .count())

        return {"ok": True, "data": count, "error": None}
    finally:
        db.close()

```

## PART 3 - Tool Contracts Under Stress

### 1. Expected input schema, rejected input, returned error output

#### a. Search\_vulnerabilities\_by\_name

##### i. Expected JSON input schema

```

1. {
    "properties": {
        "query": {
            "title": "Query",
            "type": "string"
        },
        "limit": {
            "default": 5,
            "title": "Limit",
            "type": "integer"
        }
    },
    "required": [
        "query"
    ],
    "title": "search_vulnerabilities_by_nameArguments",
    "type": "object"
}

```

- ii. Rejected JSON input
    - 1. {
 

```

              "query": "zzz",
              "limit": 5
            
```
  - iii. Returned error output
    - 1. {
 

```

              "ok": false,
              "data": null,
              "error": "No vulnerabilities found matching 'zzz'"
            
```
- b. Search\_vulnerabilities\_by\_id
  - i. Expected JSON input schema
    - 1. {
 

```

              "properties": {
                "vulnerability_id": {
                  "title": "Vulnerability Id",
                  "type": "integer"
                }
              },
              "required": [
                "vulnerability_id"
              ],
              "title": "search_vulnerabilities_by_idArguments",
              "type": "object"
            
```
  - ii. Rejected JSON input
    - 1. {
 

```

              "vulnerability_id": 999999
            
```
  - iii. Returned error output
    - 1. {
 

```

              "ok": false,
              "data": null,
              "error": "No vulnerability found with ID 999999"
            
```
- c. Count\_by\_package
  - i. Expected JSON input schema:
    - 1. {
 

```

              "properties": {
                "package_name": {
                  "title": "Package Name",

```

```

        "type": "string"
    }
},
"required": [
    "package_name"
],
"title": "count_by_packageArguments",
"type": "object"
}

```

ii. Rejected JSON input

```

1. {
    "package_name": "jelly"
}

```

iii. Returned error output

```

1. {
    "ok": false,
    "data": null,
    "error": "No package found with name 'jelly'"
}

```

## 2. Add timeout and bounded exponents.

```

from sqlalchemy import text

QUERY_TIMEOUT_MS = 5000 # MS

# Attempts to show retry behavior for Q19
TEST_RETRY_ATTEMPTS = 3

# MCP STUDIO logs must go to stderr
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def run_with_retries(func, retries=3, delay=0.1, fail_attempt=0):
    for attempt in range(retries):
        try:
            logger.info(f"Attempt {attempt + 1} of {retries}")

            if attempt < fail_attempt:
                raise RuntimeError("Simulated failure for testing retries.")
            result = func()
            logger.info(f"Attempt {attempt + 1} succeeded.")
            return result
        except Exception as e:
            logger.error(f"Attempt {attempt + 1} failed: {e}")
            if attempt < retries - 1:
                wait_time = delay * (2 ** attempt) # Exponential backoff
                logger.info(f"Retrying in {wait_time} seconds.")
                time.sleep(wait_time)
            else:
                raise

```

- Success on first attempt:

The screenshot shows the 'vulnerabilities' tool interface. On the left, the 'Tools' panel has 'search\_vulnerabilities\_by\_name' selected. The 'Results' panel displays a JSON object: 

```
1- {
2  "ok": true,
3  "data": [
4    {
5      "id": 10,
6      "vulnerability_name": "SQL Injection Vulnera
7      "vulnerability_code": "VULN-DJA-001",
8      "urgency_score": 5,
9      "package_id": 11
10     }
11  ],
12  "error": null
13 }
```

. The 'Server Console' on the right shows logs: 'Attempt 1 succeeded. domain\_server.py:33', 'Attempt 1 of 3 domain\_server.py:28', and 'Processing request of type server.py:733 CallToolRequest'.

- Failure on the first attempt followed by success after a retry.

The screenshot shows the 'vulnerabilities' tool interface. The 'Results' panel displays a JSON object: 

```
1- {
2  "ok": true,
3  "data": [
4    {
5      "id": 10,
6      "vulnerability_name": "SQL Injection Vulnera
7      "vulnerability_code": "VULN-DJA-001",
8      "urgency_score": 5,
9      "package_id": 11
10     }
11  ],
12  "error": null
13 }
```

. The 'Server Console' on the right shows logs: 'Attempt 2 succeeded. domain\_server.py:33', 'Attempt 2 of 3 domain\_server.py:28', 'Retrying in 0.1 seconds. domain\_server.py:39', 'Attempt 1 failed: Simulated failure for testing retries.', 'Attempt 1 of 3 domain\_server.py:28', and 'Processing request of type server.py:733 CallToolRequest'.

- Failure after all allowed retries, returning a clean error result instead of crashing.

The screenshot shows the 'vulnerabilities' tool interface. The 'Results' panel displays a JSON object: 

```
1- {
2  "ok": false,
3  "data": null,
4  "error": "Error occurred while search
5 }
```

. The 'Server Console' on the right shows logs: 'Attempt 3 failed: Simulated failure for testing retries.', 'Attempt 3 of 3 domain\_server.py:28', 'Retrying in 0.2 seconds. domain\_server.py:39', 'Attempt 2 failed: Simulated failure for testing retries.', 'Attempt 2 of 3 domain\_server.py:28', 'Retrying in 0.1 seconds. domain\_server.py:39', 'Attempt 1 failed: Simulated failure for testing retries.', 'Attempt 1 of 3 domain\_server.py:28', and 'Processing request of type server.py:733 CallToolRequest'.

### 3. Use VERIFY\_SEED = 264544 to simulate failures

```
VERIFY_SEED = 264544
# Simulation failures at rates of 0%, 20%, and 50% of calls, with 50 calls at each failure rate.
FAILURE_RATES = [0.0, 0.2, 0.5]
FAILURE_CALLS_COUNT = 50

REPO_ROOT = Path(__file__).resolve().parent.parent
output_dir = REPO_ROOT / "reports" / "hw05" / "raw"

def run_benchmark_call(db, rng, failure_rate):
    def operation():
        # Simulate a storage failure
        if rng.random() < failure_rate:
            raise RuntimeError("Simulated storage failure")

        # Real storage operation
        return db.query(Vulnerability).first()

    return run_with_retries(operation)

def main():
    all_results = []
    summary_results = []

    db = create_test_db()
```

| Injected failure rate | Success rate | Mean latency (ms) | p99 latency (ms) |
|-----------------------|--------------|-------------------|------------------|
| 0%                    | 100%         | 1.31ms            | 13.65ms          |
| 20%                   | 98%          | 56.88ms           | 315.39ms         |
| 50%                   | 82%          | 140.47ms          | 317.56ms         |

The retry policy is suitable for an interactive assistant, because of the high success rate. It seems like at 50% failure rate, the success rate was 82%, and the mean latency increased to 140.47ms, and the p99 latency increased to 317.56ms. At 20% injected failure rate, the success rate was 98%, showing a tradeoff that the retry policy may improve reliability, but the response times may increase due to more frequent failures.

Where longer processing times are acceptable, I would increase the number of retries for batch processing, allowing more failures to have a chance to succeed, and thus increasing the success rate.

## PART 4 - One Safe Tool Entry Point and Offline Tests

1. Create execute\_tool

```

# Part 4 Question 22
def execute_tool(name, inputs, tools=None): #temporary fixturs so tests can run offline
    try:
        if tools is None:
            tools = {
                "search_vulnerabilities_by_name" : search_vulnerabilities_by_name,
                "search_vulnerabilities_by_id" : search_vulnerabilities_by_id,
                "count_by_package" : count_by_package
            }

        # Part 5 safety rule
        if name == "search_vulnerabilities_by_name":
            query = inputs.get("query")
            if not query or not query.strip():
                return json.dumps({
                    "ok" : False,
                    "data": None,
                    "error": "There is a safety rule: vulnerability search cannot be empty"
                })

        if name not in tools:
            result = {
                "ok" : False,
                "data" : None,
                "error": f"Tool '{name}' not found."
            }
        elif name == "search_vulnerabilities_by_name":
            result = tools[name](query=inputs.get("query"), limit=inputs.get("limit", 5))
        elif name == "search_vulnerabilities_by_id":
            result = tools[name](vulnerability_id=inputs.get("vulnerability_id"))
        elif name == "count_by_package":
            result = tools[name](package_name=inputs.get("package_name"))

        return json.dumps(result)

    except Exception as e:
        return json.dumps({

```

## 2. Simple Python test runner

```

def run_tests():
    passed = 0
    total = 7 #edit baed on how many tests

    # Valid name search
    result = json.loads(
        execute_tool(
            "search_vulnerabilities_by_name",
            {"query": "Injection", "limit": 5},
            tools=test_tools
        )
    )

    assert result["ok"] is True
    print("PASS: search by name is valid")
    passed += 1

    # Rejected name search
    result = json.loads(
        execute_tool(
            "search_vulnerabilities_by_name",
            {"query": "ZZZZZZ", "limit": 5},
            tools=test_tools
        )
    )

    assert result["ok"] is False
    print("PASS: search by name is rejected")
    passed += 1

    # Valid ID search
    result = json.loads(
        execute_tool(

```

### 3. Test

```
((venv) ) (base) sangah@MacBook-Air-SAL code % {
  echo "==== HW05 Offline execute_tool Tests ====="
  echo "Start: $(date)"
  python test_execute_tool.py
  echo "End: $(date)"
  echo
} 2>&1 | tee -a ../reports/hw05/RUN_LOG.txt
==== HW05 Offline execute_tool Tests =====
Start: Sun Oct  4 18:57:57 PDT 2026
PASS: search by name is valid
PASS: search by name is rejected
PASS: search by ID - valid
PASS: search by ID is rejected
PASS: count by package is valid
PASS: count by package is rejected

6/6 tests passed
End: Sun Oct  4 18:57:58 PDT 2026

((venv) ) (base) sangah@MacBook-Air-SAL code %
```

## PART 5 - Agent Loop, Safety Rule, and Reflection

### I. Safety Rule

```
echo
) 2>&1 | tee -a ../reports/hw05/RUN_LOG.txt
==== HW05 Safety Rule Demonstration =====
Start: Sun Oct  4 19:28:12 PDT 2026
--- Allowed Call ---
[10/04/26 19:28:13] INFO Attempt 1 of 3
INFO Attempt 1 succeeded.
{"ok": true, "data": [{"id": 10, "vulnerability_name": "SQL Injection Vulnerability", "vulnerability_code": "VULN-DJA-001", "urgency_score": 5, "package_id": 11}], "error": null}
--- Blocked Call ---
{"ok": false, "data": null, "error": "There is a safety rule: vulnerability search cannot be empty"}
End: Sun Oct  4 19:28:13 PDT 2026
```

```
> test_execute_tool.py > test_search_by_id
def test_count_by_package(package_name):
    "error": None
}

return {
    "ok": False,
    "data": None,
    "error": f"No package found with name '{package_name}'"
}

test_tools = {
    "search_vulnerabilities_by_name": test_search_by_name,
    "search_vulnerabilities_by_id": test_search_by_id,
    "count_by_package": test_count_by_package
}

def run_tests():
    passed = 0
    total = 6

    # Valid name search
    result = json.loads(
        execute_tool(
            "search_vulnerabilities_by_name",
            {"query": "Injection", "limit": 5},
            tools=test_tools
        )
    )

    assert result["ok"] is True
    print("PASS: search by name is valid")
    passed += 1
```

## II. Agent Loop

```
from mcp_server.domain_server import execute_tool
from langchain_ollama import ChatOllama

MODEL_NAME = os.environ.get("SMOL_MODEL", "your-ollama-model-tag")
OLLAMA_URL = os.environ.get("OLLAMA_URL", "http://localhost:11434")

MAX_STEPS_LIMIT = 4

REPO_ROOT = Path(__file__).resolve().parent.parent
AGENT_LOG_PATH = REPO_ROOT / "reports" / "hw05" / "raw" / "agent_runs.jsonl"

def run_agent(user_input):
    model = ChatOllama(
        model=MODEL_NAME,
        temperature=0.0,
        base_url=OLLAMA_URL,
        num_ctx=2048,
    )

    messages = [
        {
            "role": "system",
            "content": """
            You help users find vulnerability information.
```

```
            """
        },
        {"role": "user", "content": user_input},
    ]

    step = 0
    while step < MAX_STEPS_LIMIT:
        step += 1
        response = model.invoke(messages)
        content = getattr(response, "content", "")

        try:
            action = json.loads(content)
        except (TypeError, json.JSONDecodeError):
            return {
                "answer": "Invalid response.",
                "history": messages
            }

        messages = [
            {
                "role": "system",
                "content": """
                You help users find vulnerability information.

                These are the available tools to utilize:
                - search_vulnerabilities_by_name(query, limit=5)
                - search_vulnerabilities_by_id(vulnerability_id)
                - count_by_package(package_name)

                To call a tool, return only:
                {"action": "tool", "tool": "tool_name", "inputs": {}}

                When all the information is available, return only:
                {"action": "final", "answer": "your answer"}
                """
            },
            {"role": "user", "content": user_input},
        ]

        step = 0
        while step < MAX_STEPS_LIMIT:
            step += 1
            response = model.invoke(messages)
            content = getattr(response, "content", "")

            try:
                action = json.loads(content)
            except (TypeError, json.JSONDecodeError):
                return {
                    "answer": "Invalid response.",
                    "history": messages
                }

            messages = [
                {
                    "role": "system",
                    "content": """
                    You help users find vulnerability information.

                    These are the available tools to utilize:
                    - search_vulnerabilities_by_name(query, limit=5)
                    - search_vulnerabilities_by_id(vulnerability_id)
                    - count_by_package(package_name)

                    To call a tool, return only:
                    {"action": "tool", "tool": "tool_name", "inputs": {}}

                    When all the information is available, return only:
                    {"action": "final", "answer": "your answer"}
                    """
                },
                {"role": "user", "content": user_input},
            ]
```

```

tool_result = execute_tool(tool_name, tool_inputs)
log_agent_step({
    "step": step,
    "tool": tool_name,
    "inputs": tool_inputs,
    "result": tool_result
})

```

### III. Additional tests

```

def run_tests():
    # Part 5 agent loop - using MockModel, stops after reaching max_steps
    original_get_model = domain_agent.get_model
    original_execute_tool = domain_agent.execute_tool

    try:
        domain_agent.get_model = lambda: MockModel()    # Use MockModel instead of Ollama

        # Replace real db tool execution
        domain_agent.execute_tool = lambda name, inputs: json.dumps({
            "ok": True,
            "data": 2,
            "error": None
        })

        result = domain_agent.run_agent("Keep checking the package.")

        assert result["stop_reason"] == "max_steps"
        assert result["step_count"] == domain_agent.MAX_STEPS_LIMIT

        print("PASS: agent stops at max_steps")
        passed += 1

    finally:
        domain_agent.get_model = original_get_model
        domain_agent.execute_tool = original_execute_tool

```

```

# Part 5 safety rule - block empty search
result = json.loads(
    execute_tool(
        "search_vulnerabilities_by_name",
        {"query": "", "limit": 5},
        tools=test_tools
    )
)

assert result["ok"] is False
assert result["data"] is None
assert "safety rule" in result["error"].lower()
print("PASS: empty vulnerability search is blocked")

passed += 1

```

#### IV. Metrics (4 Scenarios)

- ```
((venv) ) (base) sangah@MacBook-Air-SAL code % {
  echo "==== HW05 Ollama Agent Scenario ====="
  echo "Start: $(date)"
  python domain_agent.py
  echo "End: $(date)"
  echo
} 2>&1 | tee -a ../reports/hw05/RUN_LOG.txt
==== HW05 Ollama Agent Scenario =====
Start: Sun Oct  4 21:01:41 PDT 2026
Enter a vulnerability question: How many vulnerabilities are associated with requests?
[10/04/26 21:02:25] INFO      HTTP Request: POST http://localhost:11434/api/chat "HTTP/1.1 200 OK"
                    INFO      Attempt 1 of 3
                    INFO      Attempt 1 succeeded.
                    INFO      Attempt 1 of 3
                    INFO      Attempt 1 succeeded.
[10/04/26 21:02:26] INFO      HTTP Request: POST http://localhost:11434/api/chat "HTTP/1.1 200 OK"
MODEL RESPONSE: {"action":"count_by_package","inputs":{"package_name":"requests"}}
TOOL RESULT TEST: {"ok": true, "data": 2, "error": null}
MODEL RESPONSE: {"action":"final","answer":"There are 2 vulnerabilities associated with the requests package."}
{
  "answer": "There are 2 vulnerabilities associated with the requests package.",
  "step_count": 2,
  "stop_reason": "completed",
  "tool_call_count": 1
}
}
End: Sun Oct  4 21:02:26 PDT 2026
```
- ```
((venv) ) (base) sangah@MacBook-Air-SAL code % {
  echo "==== HW05 Ollama Agent Scenario ====="
  echo "Start: $(date)"
  python domain_agent.py
  echo "End: $(date)"
  echo
} 2>&1 | tee -a ../reports/hw05/RUN_LOG.txt
==== HW05 Ollama Agent Scenario =====
Start: Sun Oct  4 20:46:57 PDT 2026
Enter a vulnerability question: Find vulnerabilities with Injection in the name
[10/04/26 20:47:08] INFO      HTTP Request: POST http://localhost:11434/api/chat "HTTP/1.1 200 OK"
[10/04/26 20:47:09] INFO      Attempt 1 of 3
                    INFO      Attempt 1 succeeded.
                    INFO      HTTP Request: POST http://localhost:11434/api/chat "HTTP/1.1 200 OK"
MODEL RESPONSE: {"action":"search_vulnerabilities_by_name","tool":"search_vulnerabilities_by_name","inputs":{"query":"Injection"}}
MODEL RESPONSE: {"action":"final","answer":"The vulnerability with name 'SQL Injection Vulnerability' has been found. Its ID is 10."}
{
  "answer": "The vulnerability with name 'SQL Injection Vulnerability' has been found. Its ID is 10.",
  "step_count": 2,
  "stop_reason": "completed",
  "tool_call_count": 1
}
}
End: Sun Oct  4 20:47:10 PDT 2026
```
- ```
((venv) ) (base) sangah@MacBook-Air-SAL code % {
  echo "==== HW05 Ollama Agent Scenario ====="
  echo "Start: $(date)"
  python domain_agent.py
  echo "End: $(date)"
  echo
} 2>&1 | tee -a ../reports/hw05/RUN_LOG.txt
==== HW05 Ollama Agent Scenario =====
Start: Sun Oct  4 20:56:03 PDT 2026
Enter a vulnerability question: Find vulnerability with ID 1
[10/04/26 20:56:30] INFO      HTTP Request: POST http://localhost:11434/api/chat "HTTP/1.1 200 OK"
                    INFO      Attempt 1 of 3
                    INFO      Attempt 1 succeeded.
[10/04/26 20:56:31] INFO      HTTP Request: POST http://localhost:11434/api/chat "HTTP/1.1 200 OK"
MODEL RESPONSE: {"action":"tool","tool":"search_vulnerabilities_by_id","inputs":{"vulnerability_id":"1"}}
MODEL RESPONSE: {"action":"tool","tool":"search_vulnerabilities_by_name","inputs":{"query":"ID 1"}}
{
  "answer": "Invalid response.",
  "step_count": 2,
  "stop_reason": "invalid_json",
  "tool_call_count": 1
}
}
End: Sun Oct  4 20:56:32 PDT 2026
```

```

((venv) ) (base) sangah@MacBook-Air-SAL code % {
  echo "==== HW05 Ollama Agent Scenario ====="
  echo "Start: $(date)"
  python domain_agent.py
  echo "End: $(date)"
  echo
} 2>&1 | tee -a ../reports/hw05/RUN_LOG.txt
==== HW05 Ollama Agent Scenario =====
Start: Sun Oct  4 21:00:04 PDT 2026
Enter a vulnerability question: Search for vulnerabilities with an empty name
[10/04/26 21:00:14] INFO      HTTP Request: POST http://localhost:11434/api/chat "HTTP/1.1 200 OK"
MODEL RESPONSE: {"action":"tool","tool":"search_vulnerabilities_by_name","inputs":{}}
{
  "answer": "Invalid response.",
  "step_count": 1,
  "stop_reason": "invalid_json",
  "tool_call_count": 0
}
End: Sun Oct  4 21:00:15 PDT 2026

```

4.

## V. Reflection

```

{"step": 1, "tool": "search_vulnerabilities_by_name", "inputs": {"query": "Injection"}, "result":
  {"ok": true, "data": [{"id": 10, "vulnerability_name": "SQL Injection Vulnerability",
    "vulnerability_code": "VULN-DJA-001", "urgency_score": 5, "package_id": 11}],
  "error": null}}
{"step": 2, "stop_reason": "completed", "answer": "The vulnerability with name 'SQL Injection
  Vulnerability' has been found. Its ID is 10."}

```

When looking at this, what happened behind the scenes was that there was a model searching for the answer. The query was, “Find a vulnerability with Injection in the name.” This query then goes through the `run_agent()`, and Ollama makes a decision based on the prompt it is given. As the user prompts for a vulnerability name (not id or number of vulnerabilities associated), the model decides on making a decision by looking through the `search_vulnerabilities_by_name` tool with “Injection” as the query. The loop then interprets the JSON, and `execute_tool()` is run, which returns the matching vulnerability. The harness then returns the result to the model, and the model is able to return the final answer. The model stopped due to normal completion. So step 1, the model got the query and went through a loop and searched the database and found “SQL Injection Vulnerability”, with an ID of 10. The harness then adds this result to the messages for Ollama to retrieve. In step 2, the model returned the result and stopped as it determined that it had enough information. This resulted in the stop reason being completed, as it did not reach the `max_steps` and no safety rules were violated.